

Details on Callbacks in SROS2

This section provides a detailed overview of the callback interface in ROS2. Callbacks are referenced in the implementation section of the paper.

In ROS2, a callback is a function whose scheduling and execution are managed by ROS2's main *executor*. ROS2 callbacks fall under several categories: (1) subscription callbacks, which are used to receive and process topic-labeled data; (2) timer callbacks; (3) service and client callbacks, which are used to handle service requests on the server-side and client-side in the client/server setup of ROS2. The executor manages, schedules, and invokes these callbacks on behalf of the application. ROS2 offers both single-threaded and multi-threaded variants of the executor.

ESROS2 uses the rule-of-thumb that callbacks contain application-provided code and must therefore not be permitted to directly execute system calls. Callbacks also represent the situation wherein code in ESROS2-TCB (running in S-Dom) transfers control to application logic (running in A-Dom). In the ROS2 code base (rc1cpp in particular), the functions that invoke the callbacks are almost entirely implemented as C++ template functions, defined in headers and instantiated at compile time. ESROS2 must ensure that `mpk_exit_gate` is executed before these functions are called (so that S-Dom is write-protected before the callback is invoked), and that `mpk_entry_gate` is executed once the execution of the callback logic is completed, and control returns to ESROS2-TCB. We now detail the placement of these gates for the categories of callbacks discussed above:

(1) *Subscription callbacks.* In ROS2, the DDS middleware layer is responsible for message delivery after matching topic names. When ROS2's executor receives a new message from DDS, it invokes a function called `execute_subscription`, which in turn calls C++ templates that are instantiated as part of application logic (via a function called `handle_message`). In ESROS2-TCB, we place a `mpk_exit_gate` call just before the call to `handle_message`, thus write-protecting S-Dom. Control returns to ESROS2-TCB just following this call to `handle_message` after the callback completes, and we place a call to `mpk_entry_gate` there, representing a transition from A-Dom to S-Dom.

(2) *Timer callbacks.* Applications can register code that will be executed when a timer expires. Upon timer expiry, ROS2's executor invokes a function called `execute_any_executable`, which in turn invokes the application-provided callback via a function called `execute_callback`. As with subscription callbacks, we guard the call to `execute_callback` with the `mpk_exit_gate` gate before and `mpk_entry_gate` after the call.

(3) *Service and client callbacks.* ROS2 can also be configured to work in client/server mode. When an application is configured as a server, service callbacks are application-registered functions that handle incoming requests from clients.

Similarly, if an application is configured as a client, a client callback is application code that allows it to process replies received from a server. In the case of service callbacks, the rc1cpp function called `execute_service` invokes the application-registered callback via `handle_request`, the call to which is guarded with an exit gate/entry gate call pair. In the case of client callbacks, the corresponding rc1cpp function called `execute_client` invokes the application callback via a call to `handle_response`, which is guarded by the `mpk_exit_gate` and `mpk_entry_gate` gate pair.

In summary, ROS2's clean interfaces to execute callbacks allows us to precisely identify transition points from ESROS2-TCB to application logic, and guard these transitions between S-Dom and A-Dom with suitable gates.

Example of ESROS2-SCgate Policy

This section provides more details on the ESROS2-SCgate policies for the Franka Emika Panda Robotic Arm case study referenced in the paper. Our goal is to give an idea of the effort involved in writing policies for ESROS2-SCgate.

Figure 8 of the paper shows the number of libc calls invoked directly from various ROS2 applications in this case study as we ran the applications. Each of these calls succeeds only if the policy registered for the corresponding libc call succeeds. Figure 12 illustrates the function implementing the security policy check for the calls to `fopen64`, one of the file utilities invoked by the `robot_state_publisher` application. The policy sanity checks the input arguments to `fopen64`, in this case to check that the file(s) are opened in read-only mode, and that only the set of five files permitted by the security analyst can be opened. The calls to utilities such as `sizeof` and `strcmp` here are directly resolved to the libc code in S-Dom.

In this example, we say that `fopen64` requires five resource access checks, one each corresponding to the file (and the access mode) opened. Figure 13 presents the number of such resource access checks required for each of the libc utilities accessed by all of the applications in this case study. For example, Figure 13 shows that `move_group` accesses a total of 16 libc utilities, and `fopen64` requires 28 resource access checks. The case illustrated in the policy listing in Figure 12 is marked. Observe that the applications also accessed the generic `syscall` utility to make system calls. We found that the applications used this utility to obtain thread information by invoking `SYS_gettid`. It is unclear why these applications chose this approach instead of `gettid`. Nevertheless, we permitted it to allow the applications to function, and this is indicated in the single resource access check against the `syscall` utility.

The required resource access checks can be provided *a priori* by the security analyst, who declares what is permissible. Alternatively, a site security analyst unfamiliar with

```

static const char *fopen64_allowed_paths[] = {
    // ament index: package index
    "/opt/ros/humble/share/ament_index/resource_index/packages/urdf_parser_plugin",
    "/opt/ros/humble/share/ament_index/resource_index/packages/sdformat_urdf",
    "/opt/ros/humble/share/ament_index/resource_index/packages/urdf",

    // ament index: plugin discovery
    "/opt/ros/humble/share/ament_index/resource_index/urdf_parser_plugin__pluginlib__plugin/sdformat_urdf",
    "/opt/ros/humble/share/ament_index/resource_index/urdf_parser_plugin__pluginlib__plugin/urdf",
}; // Each of these file accesses must be permitted. We call each a resource access check, and there are 5 in this example.

bool fopen64_policy(const char *path, const char *mode) {
    if (strcmp(mode, "r") != 0) // Allow exclusively read-only opens
        return false;

    for (size_t i = 0;
         i < sizeof(fopen64_allowed_paths) / sizeof(fopen64_allowed_paths[0]);
         i++) {
        if (path_matches(path, fopen64_allowed_paths[i])) {
            return true;
        }
    }
    return false;
}

```

Figure 12. Policy for fopen64 of the robot_state_publisher application from the Robotic Arm case study in Section 6.

libc Utility	# Checks	libc Utility	# Checks
move_group		robot_state_publisher	
fclose	64	fclose	4
fcntl	4	fopen	1
fgetc	37	fopen64 ←	5
fopen	39	fwrite	1
fopen64	28	read	5
fread	38	ros2_control_node	
fseeko64	37	fclose	147
ftello64	37	fgetc	92
fwrite	2	fopen	94
mmap	4	fopen64	58
open	32	fread	105
read	29	fseeko64	108
sigaction	1	ftello64	108
socket	1	fwrite	1
socketpair	1	syscall	1
syscall	1	static_transform_publisher	
		fwrite	1
		syscall	1

Figure 13. Number of resource access checks required in the ESROS2-SCgate policies for each libc utility invoked by the ROS2 applications in the Robotic Arm case study.

case study were declared in the configurations of the applications, and the analyst can directly use the file path names (after vetting them) in the resource access checks.

The full set of resource access checks for the ESROS2-SCgate policies for each libc utility invoked by the ROS2 applications (for all three of our case studies) is available in the artifact repository associated with the paper at <https://github.com/Esros2/ESROS2>

the resources required by an application can run ESROS2-SCgate in auditing mode in a sandboxed environment, observing and vetting the resources accessed, and then including a rule for each resource accessed. A *prima facie* look at the number of resource access checks for move_group and ros2_control_node in Figure 13 would lead the reader to conclude that several tens to hundreds of resource access checks are required for some libc utilities. However, our experience (with this and other case studies) suggests that most of the resource access checks correspond directly to resource requirements specified in the configuration files of ROS2 applications. For example, all the file path names accessed by the file-related libc utilities in the ROS2 applications in this